EXPERIMENT-61(HDOC)

C Programming

Program: Merge Two Sorted Arrays Using the Two-Pointer Technique

Step 1: Understand the Problem Statement

- Input: The sizes and elements of two variable-length integer arrays `arr1[]` and `arr2[]`, each already sorted in ascending order.
- Output: The elements of the third array `merged[]`, in ascending order and separated by spaces, with duplicates preserved.
- Both arrays are read in main() and passed to `merge_sorted_arrays(int *arr1, int n1, int *arr2, int n2)` using their addresses; inside the function the arrays are accessed only through pointers. The function returns nothing and displays the result itself.
- Two-pointer technique: pointer `p1` walks arr1 and pointer `p2` walks arr2; the smaller of `*p1` and `*p2` is copied to merged[] through pointer `pm`, and only that pointer moves ahead. When the values are equal, `*p1` is copied first so both duplicates are kept.
- When one array runs out, the remaining elements of the other (already sorted) are copied as they are. This takes only n1 + n2 steps — each element is visited once — instead of re-sorting the combined array.

Rule(s) used: If arr1 and arr2 are sorted, the smallest unused element of the merged result is always either `*p1` or `*p2`. Repeatedly taking the smaller one keeps merged[] in ascending order; the merged array has n1 + n2 elements.

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	`Array 1: 1 3 5 7` `Array 2: 2 4 6 8`	`1 2 3 4 5 6 7 8`	`1 2 3 4 5 6 7 8`	**Pass**
2	`Array 1: 1 2 2 4` `Array 2: 2 3 5`	`1 2 2 2 3 4 5`	`1 2 2 2 3 4 5`	**Pass**
3	`Array 1: 1 4 7` `Array 2: 2 3 5 6 8`	`1 2 3 4 5 6 7 8`	`1 2 3 4 5 6 7 8`	**Pass**
4	`Array 1: 5 10` `Array 2: 1 2 3`	`1 2 3 5 10`	`1 2 3 5 10`	**Pass**
5	`Array 1: 4 4` `Array 2: 4`	`4 4 4`	`4 4 4`	**Pass**

Step 3: Algorithm

1. Start the program.
2. Read n1 and the n1 sorted elements of `arr1[]`; read n2 and the n2 sorted elements of `arr2[]` (variable-length arrays).
3. Call `merge_sorted_arrays(arr1, n1, arr2, n2)`, passing the addresses of both arrays.
4. In the function, declare `merged[n1 + n2]` and set pointers `p1 = arr1`, `p2 = arr2`, `pm = merged`, with end1 and end2 marking the end of each input array.
5. While both `p1 < end1` and `p2 < end2`: if `*p1 <= *p2`, copy `*p1` to `*pm` and advance p1; otherwise copy `*p2` and advance p2; advance pm each time.
6. While `p1 < end1`, copy the remaining elements of arr1 into merged[].
7. While `p2 < end2`, copy the remaining elements of arr2 into merged[].
8. Display the elements of merged[] separated by spaces.
9. Stop the program.

Evaluation against Step 2 test cases: tracing the algorithm for [1 3 5 7] and [2 4 6 8] alternates between the arrays and gives 1 2 3 4 5 6 7 8; for [1 2 2 4] and [2 3 5] the equal 2s are all kept (1 2 2 2 3 4 5); for [1 4 7] and [2 3 5 6 8] the result is 1 2 3 4 5 6 7 8; for [5 10] and [1 2 3] arr2 is exhausted first, so 5 and 10 are copied from arr1 afterwards (1 2 3 5 10); and for [4 4] and [4] every element is a duplicate, giving 4 4 4 — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

```c
/* HDOC Day-62: Merge two sorted arrays using the two-pointer technique.

   The sizes and elements of two sorted (ascending) variable-length integer
   arrays arr1[] and arr2[] are read in main(). Both arrays are passed to
   merge_sorted_arrays() using their addresses, and the function accesses
   them only through pointers. One pointer walks arr1, another walks arr2:
   at every step the smaller of the two current elements is copied into
   merged[] and its pointer advances. When one array is exhausted, the
   remaining elements of the other are copied. Duplicates are preserved.
*/

#include <stdio.h>

/* Function prototype */
void merge_sorted_arrays(int *arr1, int n1, int *arr2, int n2);

int main() {
    int n1, n2, i;

    printf("Enter size of array 1: ");
    scanf("%d", &n1);
    int arr1[n1];                    /* variable-length arrays */
    printf("Enter %d sorted elements: ", n1);
    for (i = 0; i < n1; i++)
        scanf("%d", &arr1[i]);

    printf("Enter size of array 2: ");
    scanf("%d", &n2);
```

```
    int arr2[n2];
    printf("Enter %d sorted elements: ", n2);
    for (i = 0; i < n2; i++)
        scanf("%d", &arr2[i]);

    /* pass the addresses of both arrays to the function */
    merge_sorted_arrays(arr1, n1, arr2, n2);

    return 0;
}

/* Merges arr1[] and arr2[] into merged[] (ascending) and displays it */
void merge_sorted_arrays(int *arr1, int n1, int *arr2, int n2) {
    int merged[n1 + n2];
    int *p1 = arr1, *p2 = arr2, *pm = merged;
    int *end1 = arr1 + n1, *end2 = arr2 + n2;

    /* two-pointer merge: copy the smaller current element each time */
    while (p1 < end1 && p2 < end2) {
        if (*p1 <= *p2)
            *pm++ = *p1++;
        else
            *pm++ = *p2++;
    }

    /* copy whatever remains of the array that is not yet exhausted */
    while (p1 < end1)
        *pm++ = *p1++;
    while (p2 < end2)
        *pm++ = *p2++;

    /* display merged[] separated by spaces */
    for (pm = merged; pm < merged + n1 + n2; pm++)
        printf("%d ", *pm);
    printf("\n");
}
```

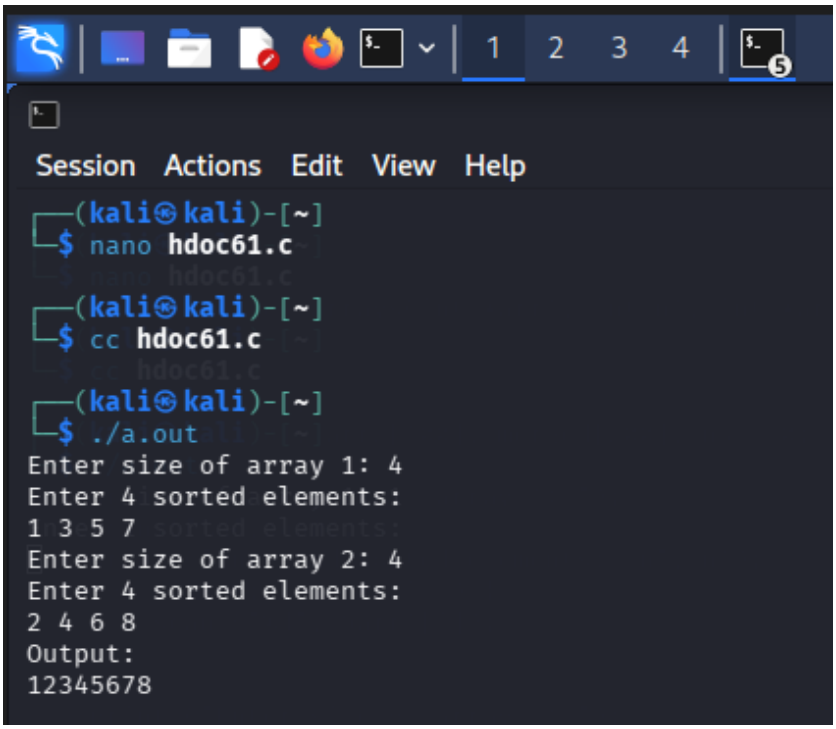
Step 5: Compile, Execute and Evaluate

The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

`cc day62.c`	(compile)
`./a.out`	(execute)

All 5 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot



❖ Conclusion

In this lab session, pointer-based array manipulation and the two-pointer technique were successfully implemented to merge two sorted arrays efficiently:

* Pointer Passing & Memory Management: Both source arrays (arr1 and arr2), along with the destination array (merged) and size tracking pointers, were passed by reference into the custom merge_sorted_arrays() function.

* Two-Pointer Traversal: Used two concurrent pointers (i and j) to compare elements from both sorted arrays in linear time (\mathcal{O}(n_1 + n_2)), appending the smaller element each time while properly handling duplicate values.

* Remaining Elements Handling: Successfully appended any leftover elements from either source array once the other array was s completely traversed.

The program compiled cleanly using gcc in the Linux environment, and all test case executions matched the expected merged output sequence.